

ThermalSense — Architecture complète

API IoT · Express.js + TypeScript · SQLite (Prisma) · JWT + RBAC · Mode dégradé · Idempotence

1. Vue globale — Flux de bout en bout

Chaque requête traverse successivement : l'injection du X-Request-ID, le rate limiter, la vérification JWT et le contrôle RBAC, avant d'atteindre le contrôleur métier.

CLIENTS	SÉCURITÉ	CONTRÔLEURS	UTILITAIRES	PERSISTANCE
Capteur IoT (DEVICE_IOT) POST /sensor/:id/measurement	① attachRequestId (X-Request-ID) ② authLoginRateLimiter 8 échecs / 10 min ③ JWT Middleware Vérif. Bearer Token Contrôle RBAC	measurement.controller Idempotency check Validation value → 201 (normal) → 202 (dégradé) → 409 (doublon)	idempotencyStore TTL 24h, Map mémoire measurementQueue Flush 5s / 5 essais	SQLite (Prisma) better-sqlite3 dev.db Measurement
Admin / Opérateur (navigateur ou API client) CRUD complet	criticalWriteRateLimiter 60 req / 15 min / IP authRefreshRateLimiter limite refresh token	building, zone, sensor actuator, alertThreshold auth (register, me) actuatorCommand	securityLogger Logs JWT / IP / route requestContext X-Request-ID trace	Building, Zone Sensor, Actuator ActuatorCommand AlertThreshold User
Lecteur (LECTEUR) GET seulement	Routes publiques : /docs (Swagger) /auth/login /auth/refresh	auth.controller POST /auth/login POST /auth/refresh scrypt + JWT signé	swagger.ts Swagger UI /docs OpenAPI 3.0	User passwordHash salt:derivedKey (hex) JWT secret (env)

2. Modèle de données

Hierarchie principale : Building → Zone → Sensor / Actuator / AlertThreshold. Les mesures sont attachées au capteur. Les utilisateurs peuvent être liés à une zone (OPERATEUR).

Entité	Champs principaux	Relations	Notes
Building	id (UUID) name address	→ Zone[] (Cascade delete)	Racine de la hiérarchie
Zone	id (UUID) name buildingId	→ Building (parent) → Sensor[] → Actuator[] → AlertThreshold[] → User[] (opérateurs)	Pivot de la gestion des droits OPERATEUR
Sensor	id (UUID) type (ex: température) status zoneId	→ Zone (parent) → Measurement[]	Peut avoir une config (PATCH /sensors/:id/config) Soft-link à une zone
Measurement	id (UUID) value (Float) timestamp (auto) sensorId	→ Sensor (parent, Cascade)	Idempotency key → 24h Mode dégradé → queue mémoire
Actuator	id (UUID) type status zoneId	→ Zone (parent) → ActuatorCommand[]	Commandes envoyées via /actuators/:id/commands
ActuatorCommand	id (UUID) command status (PENDING...) sentAt actuatorId	→ Actuator (parent, Cascade)	Statut traçable (PENDING / SENT / DONE)
AlertThreshold	id (UUID) type min (Float?) max (Float?) zoneId	→ Zone (parent)	Seuils d'alerte par zone (min et/ou max)
User	id (UUID) username (unique) passwordHash role (enum) zoneId? createdAt / updatedAt	→ Zone? (opérateur lié à une zone)	Rôles : ADMIN OPERATEUR LECTEUR DEVICE_IOT Hash : script salt:derivedKey (64 bytes)

3. Contrôle d'accès (RBAC)

Le middleware auth.middleware.ts vérifie le rôle extrait du JWT, puis pour OPERATEUR, compare le zoneId du token avec la zone ciblée par la requête.

Ressource / Action	ADMIN	OPERATEUR	LECTEUR	DEVICE_IOT
— Authentification —				
POST /auth/login	Public	Public	Public	Public
POST /auth/refresh	Public	Public	Public	Public
POST /auth/register	✓	✓ (zone propre)	—	—
GET /auth/me	✓	✓	✓	✓
— Bâtiments & Zones —				
GET /building, GET /zone	✓	✓	✓	—
POST/PATCH/DELETE /building	✓	—	—	—
POST /building/:id/zone	✓	—	—	—
— Capteurs —				
GET /sensor, GET /sensors/:id	✓	✓	✓	✓
POST /zone/:id/sensor	✓	✓ (zone propre)	—	—
PATCH /sensors/:id/config	✓	✓ (zone propre)	—	—
DELETE /sensors/:id	✓	✓ (zone propre)	—	—
— Mesures —				
GET /measurement	✓	✓	✓	—
POST /sensor/:id/measurement	✓	✓ (zone propre)	—	✓
— Actionneurs & Commandes —				
GET /actuator, GET /actuators/:id/commands	✓	✓	✓	—
POST /actuators/:id/commands	✓	✓	—	—
DELETE /actuator/:id	✓	—	—	—
— Seuils d'alerte —				
GET /alert-threshold	✓	✓	✓	✓

POST/PATCH/DELETE /alert-threshold	✓	✓ (zone propre)	—	—
------------------------------------	---	-----------------	---	---

4. Mécanismes spéciaux

Mécanisme	Déclencheur	Comportement	Limites
Idempotency Key	Header : Idempotency-Key ou body : { idempotencyKey }	1. Clé trouvée en cache → replay (Idempotent-Replayed: true) 2. Clé absente → traitement normal puis sauvegarde en cache TTL cache : 24h Cleanup auto : 1h	Stockage mémoire uniquement → reset au redémarrage
Mode dégradé	DB inaccessible ou SIMULATE_DB_DOWN=true	enqueueMeasurement() → file mémoire → 202 Accepted { queued: true, queueId } Flush automatique toutes les 5s Max 5 tentatives par entrée puis drop	File en mémoire → perte si crash serveur pendant la panne
Rate Limiting	authLoginRateLimiter authRefreshRateLimiter criticalWriteRateLimiter	Login : 8 échecs / 10 min / IP → 429 Refresh : limiteur dédié Écritures critiques : 60 req / 15 min / IP → 429	Compteurs en mémoire → reset au redémarrage
RBAC (rôle + zone) auth.middleware.ts	Chaque requête JWT vérifiée	Matrice rôle × route (17 règles) OPERATEUR : limité à sa propre zone (zoneId dans JWT) ADMIN : accès total toutes zones DEVICE_IOT : POST measurement + GET seulement	Pas de gestion multi-zones pour un OPERATEUR
Security Logger	Tout événement JWT	Logs : AUTH_VERIFY_SUCCESS / FAILED / FORBIDDEN Champs : IP, route, userId, role, audience Désactivable via JWT_LOGS=false	Logs console uniquement (pas de fichier)

5. Liste complète des endpoints

Toutes les routes sauf /docs, /auth/login et /auth/refresh requièrent un Bearer Token JWT valide.

Méthode	Route
Authentification	
POST	/auth/login
POST	/auth/refresh
POST	/auth/register
GET	/auth/me
Bâtiments	
GET	/building
POST	/building
GET	/building/:id
PATCH	/building/:id
DELETE	/building/:id
Zones	
GET	/zone
POST	/building/:id/zone
GET	/zone/:id
PATCH	/zone/:id
DELETE	/zone/:id
Capteurs	
GET	/sensor /sensors
POST	/sensors (body: { zoneId })
POST	/zone/:id/sensor
GET	/sensor/:id /sensors/:id
PATCH	/sensor/:id
DELETE	/sensor/:id /sensors/:id
GET	/sensors/:id/config
PATCH	/sensors/:id/config

Mesures	
GET	/measurement
POST	/sensor/:id/measurement /sensors/:id/measurement
Actionneurs	
GET	/actuator /actuators
POST	/zone/:id/actuator
GET	/actuator/:id
PATCH	/actuator/:id
DELETE	/actuator/:id
GET	/actuators/:id/commands
POST	/actuators/:id/commands
Commandes actionneur (détail)	
GET	/actuator/:id/commands
POST	/actuator/:id/command /actuator/:id/commands
GET	/actuator-command/:id
PATCH	/actuator-command/:id
DELETE	/actuator-command/:id
Seuils d'alerte	
GET	/alert-threshold
POST	/zone/:id/alert-threshold
PATCH	/alert-threshold/:id
DELETE	/alert-threshold/:id
Documentation	
GET	/docs (Swagger UI — public)

- Variables d'environnement clés :**
- JWT_SECRET — clé de signature JWT (défaut : "change-me" — à changer en production)
 - JWT_AUDIENCE — audience JWT (défaut : "thermalsense-api")
 - SIMULATE_DB_DOWN=true — force le mode dégradé (file d'attente + 202)
 - JWT_LOGS=false — désactive les logs d'événements JWT

- PORT — port HTTP (défaut : 3000)